Code review: Week 1

GC – Computing GC Content

고희주

Problem

Problem

The GC-content of a DNA string is given by the percentage of symbols in the string that are 'C' or 'G'. For example, the GC-content of "AGCTATAG" is 37.5%. Note that the reverse complement of any DNA string has the same GC-content.

DNA strings must be labeled when they are consolidated into a database. A commonly used method of string labeling is called **FASTA format**. In this format, the string is introduced by a line that begins with '>', followed by some labeling information. Subsequent lines contain the string itself; the first line to begin with '>' indicates the label of the next string.

In Rosalind's implementation, a string in FASTA format will be labeled by the ID "Rosalind_xxxx", where "xxxx" denotes a four-digit code between 0000 and 9999.

Given: At most 10 DNA strings in FASTA format (of length at most 1 kbp each).

Return: The ID of the string having the highest GC-content, followed by the GC-content of that string. Rosalind allows for a default error of 0.001 in all decimal answers unless otherwise stated; please see the note on **absolute error** below.

출처: rosalind.info/problems/gc

Sample Dataset

```
>Rosalind_6404
CCTGCGGAAGATCGGCACTAGAATAGCCAGAACC
GTTTCTCTGAGGCTTCCGGCCTTCCC
TCCCACTAATAATTCTGAGG
>Rosalind_5959
CCATCGGTAGCGCATCCTTAGTCCAATTAAGTCC
CTATCCAGGCGCTCCGCCGAAGGTCT
ATATCCATTTGTCAGCAGACACGC
>Rosalind_0808
CCACCCTCGTGGTATGGCTAGGCATTCAGGAACC
GGAGAACGCTTCAGACCAGCCCGGAC
TGGGAACCTGCGGGCAGTAGGTGGAAT
```

Sample Output

```
Rosalind_0808
60.919540
```

Given

FASTA 형식의 DNA 서열 최대 10개 (각 1 kbp 이하)

Return

GC content가 가장 높은 서열의 ID와 그 GC content

허용 오차 0.001

GC content (1)

- DNA 서열 전체에서 G 또는 C가 차지하는 비율(%)
  - GC content = (G 개수 + C 개수) / 전체 길이 × 100
  - 예: AGCTATAG → G, C가 3개 → 3 / 8 = 37.5%
- 역상보 서열(reverse complement)도 GC content가 같음
  - A ↔ T, G ↔ C로 짝지어 바뀌기만 하기 때문

GC content (2)

- G–C는 수소결합 3개, A–T는 2개
  - GC content가 높을수록 이중나선이 안정적 → 녹는점(Tm) 상승
- 생물종마다 GC content가 다름
  - P. falciparum 약 19%, 사람 약 41%, S. coelicolor 약 72%
  - 정체 모를 DNA가 어느 생물의 것인지 빠르게 추정하는 단서
  - 문제 제목 “Identifying Unknown DNA Quickly”의 의미

출처: Gardner et al., Nature 2002 · International Human Genome Sequencing Consortium, Nature 2001 · Bentley et al., Nature 2002

GC content (3)

- 응용
  - PCR primer 설계: 보통 GC content 40–60%를 기준으로 결합 온도 조절
  - 단백질 발현 · mRNA 의약품: codon optimization 시 GC content 함께 조절
  - NGS 데이터: GC content가 극단적인 구간의 read 편차(GC bias) 확인

FASTA format

- ‘>’로 시작하는 줄 = 서열의 ID
- 그 아래 줄들 = 하나의 서열
  - 길어서 여러 줄로 나뉘어 있을 뿐 → 다음 ‘>’ 전까지 이어 붙여야 함

```
>Rosalind_6404
CCTGCGGAAGATCGGCACTAGAATAGCCAGAACC
GTTTCTCTGAGGCTTCCGGCCTTCCC
TCCCACTAATAATTCTGAGG
>Rosalind_5959
CCATCGGTAGCGCATCCTTAGTCCAATTAAGTCC
...
```

Code review

- FASTA 파일을 한 줄씩 읽어 {ID: 서열} 딕셔너리에 저장
  - ‘>’ 줄이면 새 ID 추가, 아니면 현재 ID의 서열에 이어 붙이기
- 서열마다 (G 개수 + C 개수) / 길이 × 100 계산
- 가장 높은 값을 가진 ID와 GC content를 출력

Code review (1)

```
# ROSALIND GC - Computing GC Content
# GC content: 서열 전체 길이 중 G 또는 C가 차지하는 비율(%)

# STEP 1. FASTA 파일 읽어서 서열 모으기
seqs = {}                                   # {이름: 서열} 형태로 저장할 딕셔너리

with open("rosalind_gc.txt") as f:          # 입력 파일 열기
    for line in f:                          # 파일을 한 줄씩 읽기
        line = line.strip()                 # 줄 끝의 줄바꿈 문자 제거
        if line.startswith(">"):            # '>'로 시작하면 이름 줄
            name = line[1:]                 # '>'를 떼고 이름만 저장 (예: Rosalind_6404)
            seqs[name] = ""                 # 그 이름으로 빈 서열 칸 만들기
        else:                               # 이름 줄이 아니면 서열 줄
            seqs[name] = seqs[name] + line  # 여러 줄로 나뉜 서열을 하나로 이어 붙이기
```

Code review (2)

```
# STEP 2. 1등을 기록할 변수 만들기
top = ""                # GC content가 가장 높은 서열의 이름
top_gc = 0              # 그 서열의 GC content 값

# STEP 3. 서열마다 GC content 계산해서 가장 높은 것 찾기
for name in seqs:       # 서열을 하나씩 꺼내기
    dna = seqs[name]    # 현재 서열
    # (G 개수 + C 개수) / 전체 길이 × 100
    gc = (dna.count("G") + dna.count("C")) / len(dna) * 100
    if gc > top_gc:     # 지금까지의 1등보다 높으면
        top = name      # 1등 이름 바꾸기
        top_gc = gc     # 1등 값 바꾸기

# STEP 4. 결과 출력
print(top)              # 예: Rosalind_0808
print(top_gc)           # 예: 60.919540
```

Sample 실행 결과: Rosalind_0808 / 60.91954022988506 → 정답과 일치

다른 풀이법: Biopython (1)

- Biopython: 생물정보학 분석용 파이썬 오픈소스 라이브러리
  - FASTA 같은 서열 파일 읽기, 서열 분석 함수 등을 미리 만들어 둔 도구 모음
  - 설치: pip install biopython

```
from Bio import SeqIO                                    # 서열 파일 읽기 모듈
from Bio.SeqUtils import gc_fraction                     # GC 비율 계산 함수

records = SeqIO.parse("rosalind_gc.txt", "fasta")        # FASTA를 서열 단위로 읽기
best = max(records, key=lambda r: gc_fraction(r.seq))    # GC가 가장 높은 서열

print(best.id)                                           # ID 출력
print(gc_fraction(best.seq) * 100)                       # 0~1 비율 → % 로 변환
```

다른 풀이법: Biopython (2)

- SeqIO.parse(파일, "fasta")
  - FASTA 파일을 읽어 서열을 하나씩(record) 돌려줌
  - record.id = 서열 ID, record.seq = 서열 → 여러 줄 서열도 자동으로 합쳐 줌
- gc_fraction(서열)
  - GC content를 0~1 사이 비율로 계산 → × 100 해야 %
- max(records, key=lambda r: gc_fraction(r.seq))
  - 각 서열의 GC 비율을 기준으로 가장 큰 서열 하나를 고름
  - 직접 구현의 top, top_gc 비교 과정을 한 줄로 처리
- 직접 구현 vs Biopython
  - 직접 구현: FASTA 구조와 계산 원리를 이해하기 좋음
  - Biopython: 코드가 짧고, 파일 읽기 실수가 적음 → 실제 분석에서 주로 사용